ASSIGNMENT

Advanced DBMS and Administration

(Paper 1T4)

First Year M.C.A. — Semester I (CBCS)

RTM Nagpur University

Name: _____

Roll No: _____

Q1. Explain Function dependency in details. (Unit 1)

A **Functional Dependency (FD)** is a constraint between two sets of attributes of a relation. X → Y means that the value of X **uniquely determines** the value of Y — two rows with the same X value must have the same Y value. X is the **determinant** and Y is the **dependent**. FDs are the basis of normalization.

- **Example:** In STUDENT(Roll_No, Name, Course), Roll_No → Name, because one roll number belongs to only one student.
- **Trivial FD:** X → Y where Y is a subset of X, e.g. {Roll_No, Name} → Name.
- **Non-trivial FD:** X → Y where Y is not a subset of X, e.g. Roll_No → Name.
- **Full functional dependency:** Y depends on the whole of a composite key, not on a part of it, e.g. {Roll_No, Subject} → Marks.
- **Partial dependency:** Y depends on only a part of a composite key, e.g. {Roll_No, Subject} → Name (Name depends only on Roll_No). It is removed in 2NF.
- **Transitive dependency:** X → Y and Y → Z, so X → Z indirectly, e.g. Roll_No → Dept_No → Dept_Name. It is removed in 3NF.
- **Multivalued dependency (X →→ Y):** one X value has a set of Y values independent of other attributes. It is removed in 4NF.
- **Armstrong's Axioms:** Reflexivity (if Y ⊆ X then X → Y), Augmentation (if X → Y then XZ → YZ), Transitivity (if X → Y and Y → Z then X → Z). They are used to find all FDs (closure) from a given set.

Conclusion: Functional dependencies describe how attributes depend on each other. They help to identify keys, find redundancy and normalize the database into a good design.

Q2. What is normalization? Explain Different forms with example. (Unit 1)

Normalization is the process of organizing the tables of a database to **reduce data redundancy** and avoid **insertion, update and deletion anomalies**. A large table is decomposed into smaller tables using functional dependencies, step by step, through normal forms.

- **1NF (First Normal Form):** Every column holds **atomic (single) values**; no repeating groups. Example: a Phone column holding "98xx, 97xx" is split so that each row has one phone number.
- **2NF (Second Normal Form):** In 1NF and **no partial dependency** — every non-key attribute depends on the whole primary key. Example: in (Roll_No, Subject, Marks, Name), Name depends only on Roll_No, so it is moved to STUDENT(Roll_No, Name).
- **3NF (Third Normal Form):** In 2NF and **no transitive dependency** — non-key attributes depend only on the key. Example: in (Emp_No, Dept_No, Dept_Name), Dept_Name depends on Dept_No, so it is moved to DEPT(Dept_No, Dept_Name).
- **BCNF (Boyce-Codd Normal Form):** A stronger 3NF — for every FD X → Y, **X must be a super key**. Example: in (Student, Subject, Teacher) where Teacher → Subject, Teacher is not a key, so the table is split into (Student, Teacher) and (Teacher, Subject).
- **4NF (Fourth Normal Form):** In BCNF and **no multivalued dependency**. Example: (Student, Hobby, Language) is split into (Student, Hobby) and (Student, Language).
- **5NF (Fifth Normal Form / PJNF):** In 4NF and **no join dependency** — the table cannot be decomposed further without losing data on joining.
- Decomposition must be **lossless** (the original table can be rebuilt by joining) and preferably **dependency preserving**.

Conclusion: Normalization gives a clean database design with minimum redundancy and no anomalies. In practice most databases are normalized up to 3NF or BCNF.

Q3. Explain different type of keys. (Unit 1)

A **key** is an attribute or a set of attributes that **uniquely identifies a row (tuple)** in a table. Keys are also used to create relationships between tables and to maintain integrity.

- **Super Key:** Any set of attributes that uniquely identifies a row. Example: {Roll_No}, {Roll_No, Name}.
- **Candidate Key:** A **minimal super key** — no attribute can be removed from it. A table can have many candidate keys. Example: Roll_No, Email.
- **Primary Key:** The candidate key chosen to identify rows. It cannot be **NULL** and must be **unique**. Example: Roll_No.
- **Alternate (Secondary) Key:** Candidate keys that are not chosen as the primary key. Example: Email.
- **Foreign Key:** An attribute of one table that refers to the primary key of another table; it maintains **referential integrity**. Example: Dept_No in EMP refers to DEPT(Dept_No).
- **Composite Key:** A key made of **two or more attributes**. Example: {Roll_No, Subject_Code} in a MARKS table.
- **Unique Key:** Values must be unique, but **one NULL is allowed** (unlike the primary key).
- **Surrogate Key:** A system-generated key with no business meaning, e.g. an auto-increment ID or sequence number.

Conclusion: Keys identify records uniquely and link tables together. Choosing proper primary and foreign keys is essential for data integrity in a relational database.

Q4. What is query processing? Explain Query processing stages in details. (Unit 1)

Query processing is the set of activities the DBMS performs to **translate a high-level SQL query into an efficient execution plan** and run it to get the result. The aim is to give the correct result with the minimum cost (disk I/O and CPU time).

- **1. Parsing and Translation:** The parser checks the **syntax** of the query and verifies that tables and columns exist (semantic check using the data dictionary). The query is translated into an internal form — a **relational algebra expression / query tree**.
- **2. Query Optimization:** The optimizer creates **equivalent expressions** (e.g. pushing selections down, reordering joins) and chooses the **cheapest execution plan** using statistics such as number of rows, indexes and selectivity. Methods: rule-based (RBO) and cost-based (CBO).
- **3. Query Execution Plan:** The chosen plan specifies the algorithm for each operation — full table scan or index scan, nested-loop / sort-merge / hash join, and the order of operations.
- **4. Query Evaluation (Execution):** The execution engine runs the plan, reads the data blocks and returns the result to the user.
- **Cost estimation:** Cost is measured mainly by the number of **disk block accesses**, plus CPU and memory usage.
- In Oracle, the plan can be seen using **EXPLAIN PLAN**.

Conclusion: Query processing converts SQL into an optimized plan and executes it. Good optimization can reduce the execution time of a query from minutes to seconds.

Q5. Explain method for joining tables. (Unit 1)

A **join** combines rows from two or more tables based on a related column. The query optimizer chooses a **join method (algorithm)** depending on table size, available indexes and memory, because the join is one of the most expensive operations.

- **Nested Loop Join:** For **each row of the outer table**, the whole inner table is scanned to find matching rows. Simple; best when the outer table is small and the inner table has an **index** on the join column. Cost is high for large tables.
- **Block Nested Loop Join:** Works block by block instead of row by row, which reduces the number of disk reads.
- **Index Nested Loop Join:** Uses an index on the inner table to find matching rows directly instead of scanning it fully.
- **Sort-Merge Join:** Both tables are **sorted on the join column** and then merged in one pass. Good for large tables and non-equality joins (<, >).
- **Hash Join:** A **hash table** is built on the smaller table using the join column, then the larger table is scanned and probed against it. Fastest for large tables with equality (=) joins.
- **Multiple (Multi-way) Join:** When more than two tables are joined, the optimizer decides the **join order**, joining two tables at a time and using the result for the next join.

Conclusion: Nested loop is best for small or indexed tables, sort-merge for sorted data, and hash join for large tables. The optimizer selects the cheapest method automatically.

Q6. What is Serializibility? Explain type of serializibility. (Unit 2)

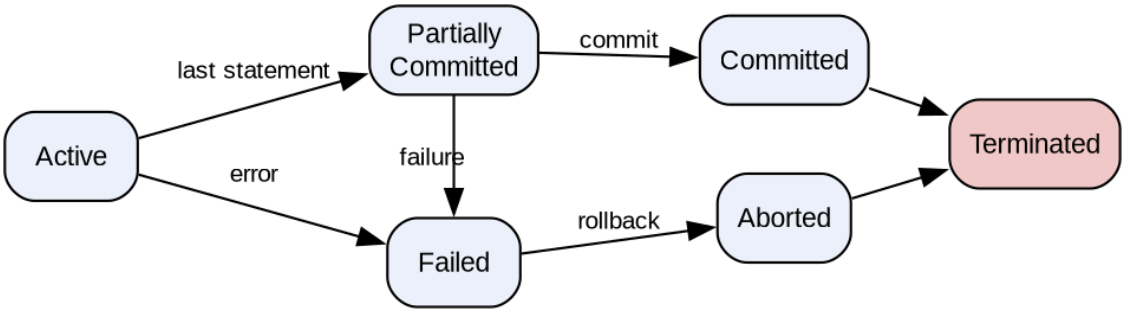
When many transactions run **concurrently**, their operations are interleaved. **Serializability** means that a concurrent (non-serial) schedule produces the **same result as some serial schedule** (transactions one after another). A serializable schedule keeps the database consistent.

- **Serial schedule:** Transactions execute one after another with no interleaving — always consistent but slow.
- **Conflicting operations:** Two operations conflict if they belong to different transactions, access the same data item, and at least one is a **write** (R-W, W-R, W-W).
- **1. Conflict Serializability:** A schedule is conflict serializable if it can be converted into a serial schedule by **swapping non-conflicting operations**.
- **Testing:** Draw a **precedence graph** — a node for each transaction and an edge Ti → Tj when an operation of Ti conflicts with a later operation of Tj. If the graph has **no cycle**, the schedule is conflict serializable.
- **2. View Serializability:** A schedule is view equivalent to a serial schedule if it has the same **initial reads, the same reads-from relations and the same final writes**.
- Every conflict serializable schedule is view serializable, but not the reverse (view serializability allows **blind writes**).
- Serializability is ensured in practice using **locking (2PL)** or **timestamp ordering** protocols.

Conclusion: Serializability is the main correctness criterion for concurrent transactions. Conflict serializability is easy to test with a precedence graph, so it is used in practice.

Q7. Explain state of transaction with diagram. (Unit 2)

A **transaction** is a logical unit of work (a set of operations such as read and write) that must execute completely or not at all, following the **ACID** properties. During its life, a transaction passes through different **states**.



- **Active:** The initial state; the transaction is executing its read/write operations.
- **Partially Committed:** The last statement has executed, but the changes are still in main memory (buffer), not yet permanently saved.
- **Committed:** All changes are permanently written to the database after a successful check. A committed transaction cannot be rolled back.
- **Failed:** Normal execution cannot continue because of an error, system crash or violation of a constraint.
- **Aborted:** The transaction is **rolled back** and the database is restored to the state before the transaction. The system can then **restart** or **kill** the transaction.
- **Terminated:** The transaction leaves the system after commit or abort.

Conclusion: The transaction states show how the DBMS guarantees atomicity — a transaction either commits completely or is aborted and rolled back, so the database always stays consistent.

Q8. Explain schedules and types. (Unit 2)

A **schedule** is the **chronological order** in which the operations (read, write, commit, abort) of multiple concurrent transactions are executed. The order of operations within each transaction is kept the same.

- **Serial Schedule:** Transactions run **one after another** completely, e.g. all of T1 then all of T2. Always consistent, but no concurrency.
- **Non-Serial (Concurrent) Schedule:** Operations of transactions are **interleaved**. Gives better throughput but may cause inconsistency.
- **Serializable Schedule:** A non-serial schedule whose result is equal to some serial schedule — types: **conflict serializable** and **view serializable**.
- **Recoverable Schedule:** If Tj reads data written by Ti, then **Ti commits before Tj**. Recovery is possible after a failure.
- **Non-Recoverable Schedule:** Tj reads Ti's data and commits before Ti; if Ti then fails, Tj cannot be undone.
- **Cascading Schedule:** The failure of one transaction forces rollback of other transactions that read its uncommitted data (**cascading rollback**).
- **Cascadeless Schedule:** Transactions read only **committed** data, so cascading rollback does not occur.
- **Strict Schedule:** A data item written by a transaction cannot be read or written by another transaction until the first one commits or aborts. Easiest to recover.

Conclusion: Schedules decide the correctness of concurrent execution. A good DBMS allows only schedules that are serializable and recoverable (preferably cascadeless or strict).

Q9. Describe deadlock in details. (Unit 2)

A **deadlock** is a situation in which two or more transactions are **waiting for each other to release locks**, so none of them can continue. Example: T1 locks A and waits for B, while T2 locks B and waits for A.

- **Necessary conditions (all four must hold):** Mutual exclusion, Hold and wait, No pre-emption, and Circular wait.
- **Deadlock Prevention:** Design the protocol so that deadlock never occurs — e.g. lock all data items in advance, or lock items in a fixed order.
- **Wait-Die scheme (non-preemptive):** An older transaction may wait for a younger one; a younger transaction requesting a lock held by an older one **dies** (rolls back) and restarts with the same timestamp.
- **Wound-Wait scheme (preemptive):** An older transaction **wounds** (rolls back) the younger one holding the lock; a younger transaction waits for an older one.
- **Timeout:** A transaction that waits longer than a fixed time is rolled back.
- **Deadlock Detection:** The DBMS maintains a **wait-for graph** (edge Ti → Tj when Ti waits for Tj). A **cycle** in the graph means a deadlock.
- **Recovery:** Choose a **victim** transaction (the one with least cost — least work done, fewest locks), **roll it back** and restart it. Avoid starvation by not choosing the same victim again and again.
- Oracle detects deadlocks automatically and reports **ORA-00060**, rolling back one statement.

Conclusion: Deadlocks cannot always be avoided in locking systems, so a DBMS uses prevention, timeouts or detection with victim rollback to keep transactions running.

Q10. What is Application Partitioning? Explain its types. (Unit 2)

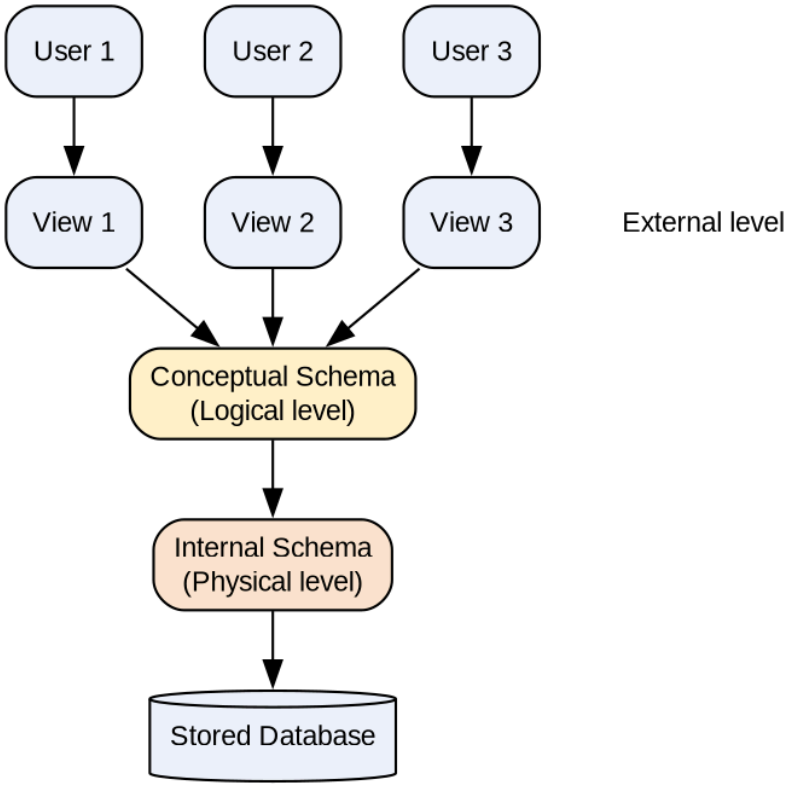
In client/server computing, **application partitioning** means **dividing an application into logical parts (components) and placing them on different machines** — client, application server or database server — to get the best performance, scalability and easy maintenance.

- **Components of an application:**
- **Presentation logic** — user interface (input/output screens).
- **Business (processing) logic** — rules and calculations of the application.
- **Data (storage) logic** — data storage, retrieval and integrity.
- **Two-tier partitioning:** The client holds the presentation and most business logic (**fat client**); the server holds the database. Simple but hard to maintain for many users.
- **Thin-client two-tier:** Business logic is moved to the database server as **stored procedures and triggers**; the client handles only the interface.
- **Three-tier partitioning:** Presentation on the client, business logic on a separate **application server**, data on the **database server**. Scalable, secure and easy to update.
- **N-tier partitioning:** Business logic is further divided among several servers (web server, application servers, services) for very large systems.
- **Advantages:** load balancing, better performance, reusability of components, security, and easier maintenance.

Conclusion: Application partitioning places each part of an application on the most suitable machine. Three-tier and n-tier partitioning are widely used in modern web and enterprise applications.

Q11. Explain DBMS architecture. (Unit 2)

DBMS architecture describes how a database system is organized and how users interact with the data. The standard is the **ANSI/SPARC three-schema architecture**, which separates the user's view from the physical storage to provide **data independence**.



- **External Level (View level):** The highest level; describes the part of the database that a particular user or application sees (**user views**). Different users can have different views.
- **Conceptual Level (Logical level):** Describes **what data** is stored and the relationships among them — all tables, attributes, constraints — for the whole database. Defined by the DBA.
- **Internal Level (Physical level):** Describes **how data is physically stored** on disk — files, indexes, blocks, access paths.
- **Mappings:** External/conceptual mapping and conceptual/internal mapping convert requests between levels.
- **Logical data independence:** The conceptual schema can be changed without changing the external views.
- **Physical data independence:** The internal schema can be changed (e.g. adding an index) without changing the conceptual schema.
- **Tier architectures:** 1-tier (DB on the same machine), 2-tier (client ↔ DB server), 3-tier (client ↔ application server ↔ DB server).

Conclusion: The three-level architecture hides storage details from users and provides logical and physical data independence, which makes the database flexible and easy to maintain.

Q12. Explain the concept of buffer management. (Unit 3)

Disk access is much slower than memory access. **Buffer management** is the part of the DBMS that manages the **buffer pool** — an area of main memory that holds copies of disk blocks — so that most data requests are served from memory instead of the disk.

- **Buffer pool:** Divided into **frames**, each holding one disk block (page).
- **Buffer manager:** When a block is requested, it checks the buffer. If the block is present (**hit**), it is returned directly; if not (**miss**), it is read from disk into a free frame.
- **Replacement policy:** When no frame is free, a block is chosen for replacement — **LRU** (Least Recently Used), MRU, FIFO or Clock algorithms. Oracle uses an LRU-based algorithm.
- **Pinned blocks:** A block being used by a transaction is **pinned** and cannot be replaced.
- **Dirty blocks:** Modified blocks are marked dirty and must be **written back to disk** before replacement (by DBWR in Oracle).
- **Forced output:** Blocks are written to disk at checkpoints or when required for recovery.
- **Write-Ahead Logging (WAL):** The log records of a block must be written to disk before the block itself, to make recovery possible.
- **Hit ratio:** (Buffer hits / total requests) × 100 — a high ratio means good performance.

Conclusion: Buffer management reduces disk I/O by keeping frequently used blocks in memory. A proper buffer size and replacement policy greatly improve database performance and support recovery.

Q13. Write about tablespace in crash recovery. (Unit 3)

In Oracle, a **tablespace** is a logical storage unit made of one or more physical **datafiles**. When a crash or media failure damages datafiles, recovery can be done **tablespace by tablespace**, so the rest of the database can stay available.

- **Instance (crash) recovery:** After a power failure or instance crash, Oracle automatically performs recovery at the next startup — **roll forward** using redo log files and **roll back** uncommitted transactions using undo. No DBA action is needed.
- **Media recovery:** When a datafile of a tablespace is lost or corrupted, the DBA restores it from **backup** and applies **archived and online redo logs**.
- **Taking a tablespace offline:** ALTER TABLESPACE users OFFLINE IMMEDIATE; — the damaged tablespace is taken offline while other tablespaces remain online.
- **Restore and recover:** Restore the datafile from backup, then RECOVER TABLESPACE users; and bring it back with ALTER TABLESPACE users ONLINE;
- **SYSTEM and UNDO tablespaces** cannot be taken offline; their failure needs recovery with the database in **MOUNT** mode.
- **Tablespace Point-In-Time Recovery (TSPITR):** Recovers one or more tablespaces to an earlier time (e.g. before a wrong DROP), without affecting other tablespaces.
- **Backup mode:** ALTER TABLESPACE users BEGIN BACKUP / END BACKUP; allows an online (hot) backup of a tablespace's datafiles in ARCHIVELOG mode.
- **RMAN** commands: RESTORE TABLESPACE users; RECOVER TABLESPACE users;

Conclusion: Tablespaces make crash and media recovery flexible — only the damaged tablespace is restored and recovered while the remaining database keeps working.

Q14. What is backup in oracle? Explain recovery strategies. (Unit 3)

A **backup** is a copy of the database (datafiles, control files, archived redo logs) used to **reconstruct the database after a failure** such as disk crash, user error or corruption. **Recovery** is the process of restoring the backup and applying redo to bring the database to a consistent state.

Types of Backup

- **Physical backup:** Copies of the physical files. **Cold (offline/consistent)** backup is taken after shutting down the database; **Hot (online/inconsistent)** backup is taken while the database is open in **ARCHIVELOG** mode.
- **Logical backup:** Copies of database objects (tables, schemas) using the **Export (exp / expdp)** utility; restored with **Import (imp / impdp)**.
- **RMAN backup:** Recovery Manager takes full or **incremental** backups and tracks them automatically.

Recovery Strategies

- **Instance recovery:** Automatic after an instance crash — roll forward with redo logs, roll back uncommitted transactions.
- **Complete media recovery:** Restore the lost datafiles and apply **all** archived and online redo logs — no committed data is lost.
- **Incomplete (Point-in-Time) recovery:** Recover up to a specific time, SCN or log sequence (e.g. before an accidental DROP TABLE); the database is opened with RESETLOGS.
- **Logical recovery:** Import the exported table or schema.
- **Flashback:** Flashback Table/Query/Database undoes user errors quickly without restoring backups.
- **Standby database:** A copy of the database kept updated with redo; it takes over if the primary fails (Data Guard).

Conclusion: A good backup strategy (regular RMAN backups + ARCHIVELOG mode + exports) together with the right recovery method ensures that an Oracle database can be recovered with minimum data loss and downtime.

Q15. Explain control file and redo log files. (Unit 3)

An Oracle database is made of three types of physical files: **datafiles, control files and redo log files**. Control files and redo log files are essential for starting the database and for recovery.

Control File

- A small **binary file** that records the **physical structure** of the database.
- Contains: database name and creation time, names and locations of **datafiles and redo log files**, current log sequence number, **checkpoint** information and RMAN backup details.
- Oracle reads it when the database is **mounted**; without it, the database cannot open.
- It is **multiplexed** (2–3 copies on different disks) for safety — parameter CONTROL_FILES.
- View: SELECT name FROM v$controlfile;

Redo Log Files

- Record **all changes** made to the database (redo entries), so that changes can be **re-applied** during recovery.
- The **LGWR** process writes redo entries from the redo log buffer to the online redo log files.
- There are at least **two redo log groups**, used in a **circular** way; when one is full, a **log switch** occurs to the next.
- Each group can have multiple **members** (copies on different disks) — multiplexing.
- In **ARCHIVELOG** mode, filled redo logs are copied to **archived redo logs** by the ARCn process, allowing complete recovery.
- View: SELECT * FROM v$logfile; SELECT * FROM v$log;

Conclusion: The control file tells Oracle where all database files are, and the redo log files record every change. Together they make instance and media recovery possible, so both should always be multiplexed.

Q16. Explain buffer cache and redo logs buffer. (Unit 4)

The **SGA (System Global Area)** is the shared memory area of an Oracle instance. Two of its most important parts for performance are the **database buffer cache** and the **redo log buffer**.

Database Buffer Cache

- Holds **copies of data blocks** read from the datafiles. All users share it.
- If a requested block is in the cache (**cache hit**), it is read from memory; otherwise (**cache miss**) it is read from disk.
- Modified blocks are called **dirty buffers**; the **DBWR** process writes them to datafiles.
- Managed using an **LRU (Least Recently Used)** list; free buffers are reused.
- Size: DB_CACHE_SIZE; can have KEEP and RECYCLE pools.
- **Tuning:** Buffer cache hit ratio should be high (above 90%) — checked with v$sysstat / v$db_cache_advice.

Redo Log Buffer

- A **circular buffer** that stores **redo entries** (records of all changes) before they are written to the online redo log files.
- The **LGWR** process writes the buffer to disk on **COMMIT**, every 3 seconds, when it is one-third full, or before DBWR writes.
- Size: LOG_BUFFER parameter.
- **Tuning:** The statistic **"redo log space requests"** (v$sysstat) should be near zero; if it is high, increase LOG_BUFFER or use faster disks.

Conclusion: The buffer cache reduces disk reads of data, and the redo log buffer makes logging of changes fast. Proper sizing of both is a key step in Oracle performance tuning.

Q17. Explain database integrity. (Unit 4)

Database integrity means the **accuracy, consistency and validity** of the data stored in a database throughout its life. The DBMS enforces integrity using **integrity constraints** — rules that every insert, update and delete must follow.

- **Need:** prevents wrong or invalid data entry, keeps related tables consistent, and makes the data reliable for decisions.
- **Domain integrity:** Values of a column must come from a valid domain (datatype, range, format) — enforced by datatype, **CHECK** and **DEFAULT** constraints. Example: CHECK (age >= 18).
- **Entity integrity:** Every table must have a **primary key** whose value is **unique and NOT NULL**, so each row is identified.
- **Referential integrity:** A **foreign key** value must match an existing primary key value in the referenced table, or be NULL. Options: ON DELETE CASCADE / SET NULL.
- **Key / Unique constraint:** Values in a UNIQUE column cannot be repeated.
- **NOT NULL constraint:** A column must always have a value.
- **User-defined (business) integrity:** Business rules enforced using **triggers**, stored procedures or assertions. Example: salary cannot decrease.
- **Example:** CREATE TABLE emp (empno NUMBER PRIMARY KEY, ename VARCHAR2(20) NOT NULL, sal NUMBER CHECK (sal > 0), deptno NUMBER REFERENCES dept(deptno));

Conclusion: Integrity constraints automatically protect the database from invalid data. Defining them at the database level is safer than checking only in application programs.

Q18. Database configuration and I/O issues. (Unit 4)

Disk **I/O (input/output)** is the slowest part of a database system. **Database configuration** for I/O means placing and sizing Oracle files and memory so that disk reads and writes are **minimized and evenly distributed**, avoiding I/O bottlenecks (contention).

- **Separate datafiles and redo logs:** Place redo log files on different disks from datafiles, because LGWR writes continuously and sequentially.
- **Separate tables and indexes:** Keep tables and their indexes in **different tablespaces on different disks** so they can be read in parallel.
- **Distribute I/O (striping):** Spread datafiles across many disks using **RAID** or Oracle **ASM**, so no single disk becomes a hot spot.
- **Separate SYSTEM, UNDO and TEMP tablespaces:** Avoid mixing user data with system, undo and sort data.
- **Multiplex control and redo files** on different disks for safety without hurting performance.
- **Archive logs** on a separate disk so archiving does not slow the online redo logs.
- **Proper memory sizing:** A larger buffer cache and SORT_AREA_SIZE / PGA reduce physical reads and disk sorts.
- **Block size and extents:** Choose a suitable DB_BLOCK_SIZE and use locally managed tablespaces to reduce fragmentation and row chaining.
- **Monitoring:** Use **v$filestat**, v$tempstat, AWR/Statspack reports and alert/trace files to find disks and files with high I/O.

Conclusion: Good database configuration — separating and balancing files across disks and sizing memory properly — removes I/O bottlenecks and is one of the most effective ways to tune Oracle performance.